

Casos de Uso e Testes

Trabalho 01, Programação Paralela e Distribuída, UFSC 2026.1.

Membros do grupo: Estéfano Tuyama, Gean Pereira, Tom Sales

Este documento descreve os casos de uso utilizados para validar o comportamento do sistema, com explicação das saídas esperadas e conclusões.

Caso 1: Curva de speedup variando o número de consumers

Objetivo:

Demonstrar empiricamente que o workload é I/O-bound e que aumentar o número de threads consumidoras escala o throughput linearmente, mesmo excedendo o número de núcleos físicos da máquina.

Procedimento:

Rodar a simulação algumas vezes mantendo `numberOfOrders = 100` e `errorRate = 0`, variando apenas `numberOfConsumerThreads`. Cronometrar o tempo total entre o início do main e o retorno do último `awaitTermination` (usar `System.nanoTime()` no início e no fim).

Execução	<code>numberOfConsumerThreads</code>
1	1
2	5
3	25
4	50

Saída:

Consumers	Tempo (s)	Speedup
1	~300	1x
5	~60	5x
25	~12	25x
50	~6	50x

Conclusão:

O speedup é quase linear até aproximadamente 50 consumers, ponto em que cada thread processa apenas 1 pedido e não há mais trabalho a paralelizar. A curva valida:

1. O workload é I/O-bound (dominado pelos `Thread.sleep` que simulam APIs externas).
2. Threads em estado de espera não competem por núcleos físicos, permitindo que o número de threads supere o número de cores sem perda de eficiência.
3. A regra "threads igual a núcleos" não se aplica a este tipo de workload.

Caso 2: Distribuição de status por taxa de erro

Objetivo:

Validar que o pipeline de três estágios (validação de dados, validação financeira, logística) aplica a taxa de erro de forma independente em cada etapa, e demonstrar o efeito multiplicativo de falhas em pipeline.

Procedimento:

Adicionar contadores ao Main (um AtomicInteger por status, ou um ConcurrentHashMap<OrderStatus, AtomicInteger>) que são incrementados ao final do processamento de cada pedido. Imprimir o resumo após o shutdown completo. Rodar 3 vezes com numberOfOrders = 1000 e numberOfConsumerThreads = 25 , variando apenas errorRate.

Execução	errorRate
1	0%
2	25%
3	50%

Saída:

errorRate	DELIVERED	FAILED_VALIDATION	FAILED_FINANCIAL	FAILED_LOGISTICS
0%	1000	0	0	0
25%	~422	~250	~187	~140
50%	~125	~500	~250	~125

Conclusão

Os valores observados convergem para os teóricos (com variação estatística inerente ao Math.random). A distribuição valida que cada estágio aplica a taxa de erro de forma independente, e que pipelines com múltiplos pontos de falha sofrem degradação multiplicativa: com 25% de erro por etapa, menos da metade dos pedidos completam o fluxo.